

SE Lab 1: Requirements Engineering and UML Use-Case Modelling

Problem Statement #31 - Multi-Vendor Artisan E-Commerce Marketplace

SRN: PES1UG24AM390

1. Requirements Table

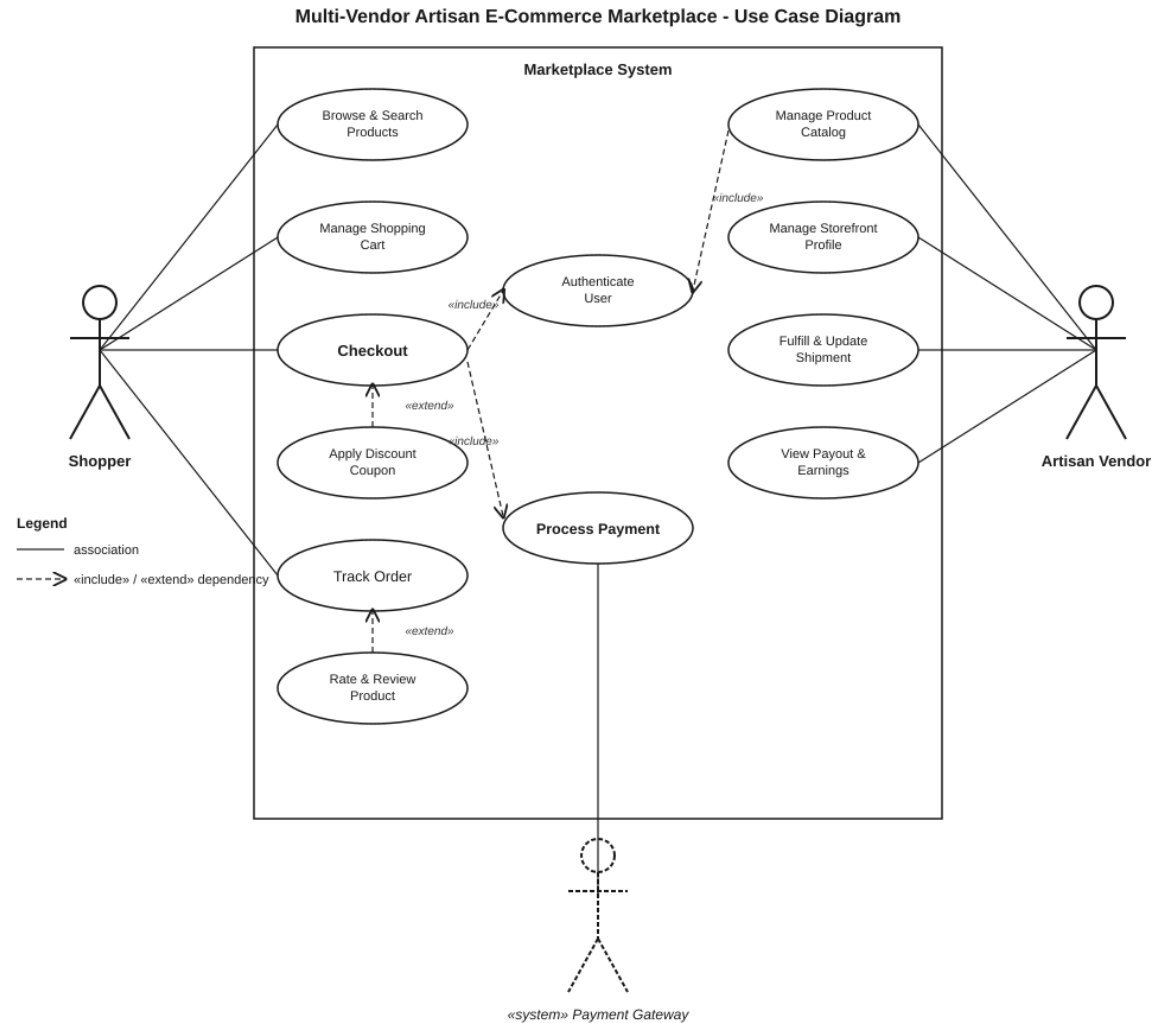
1.1 Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria	Rationale
FR-001	Functional - Payments	The system shall split a shopper's cart payment at checkout, allocating each vendor's item earnings to their vendor account after deducting a 5% platform commission, and shall generate an itemized payout receipt per vendor sub-order.	High	Pass: sum of all vendor payouts + platform commission equals the cart total (to 2-decimal accuracy). Fail: any payout discrepancy, or a payout not credited within the same transaction.	The core trust promise of a multi-vendor marketplace is accurate, automatic settlement between many independent artisans without manual reconciliation.
FR-002	Functional - Catalog Management	The system shall let an authenticated Artisan Vendor create, update, publish, and remove product listings (title, description, price, stock, images) within their own storefront only.	High	Pass: a vendor's catalog changes appear in the shopper-facing view within 2 seconds and are attributed only to that vendor's storefront. Fail: changes are lost, delayed beyond threshold, or attributed to the wrong storefront.	Vendors are independent craftspeople managing their own inventory; self-service catalog control removes the platform as a bottleneck.
FR-003	Functional - Search and Discovery	The system shall let a Shopper search the aggregated multi-vendor catalog by keyword and filter/sort results by category, price range, and vendor rating.	Medium	Pass: results include only currently in-stock items matching applied filters, ranked by relevance/rating. Fail: out-of-stock or non-matching items appear, or a filter is silently ignored.	With many independent vendors, shoppers need effective discovery tools to find relevant handmade goods in a large, decentralized catalog.
FR-004	Functional - Order Management	The system shall let a Shopper place a single checkout spanning multiple vendors, automatically splitting it into one sub-order per vendor, and shall let the shopper track each sub-order's fulfillment status independently.	High	Pass: one multi-vendor cart yields correctly partitioned, independently trackable sub-orders. Fail: sub-orders are merged incorrectly, or one vendor's status update affects another vendor's sub-order.	A single cart can span several artisans who ship independently, so fulfillment must be tracked per vendor, not per cart.
FR-005	Functional - Order Fulfillment	The system shall let an Artisan Vendor view sub-orders assigned to them, update shipment status (Processing - Shipped - Delivered), and attach tracking information.	Medium	Pass: status changes are timestamped, visible to the shopper in real time, and restricted to the vendor's own sub-orders. Fail: a vendor can view or edit another vendor's sub-order, or a status change isn't reflected to the shopper.	Fulfillment is decentralized to independent artisans, so the platform needs a controlled channel for vendors to report progress without exposing other vendors' data.

1.2 Non-Functional Requirements

ID	Type	Description	Priority	Acceptance Criteria	Rationale
NFR-001	Performance	The product catalog shall support high-resolution image rendering with CDN caching, delivering image load times under 500 ms even under peak concurrent traffic.	High	Pass: load testing confirms under 500 ms image render latency at simulated peak load (e.g. 1,000 concurrent users). Fail: latency exceeds the threshold or images fail to load under load.	Artisan products are highly visual; slow-loading images directly hurt conversion and undercut the seller's ability to showcase craftsmanship.
NFR-002	Security and Reliability	All payment and payout transactions shall be encrypted in transit (TLS 1.2+) and at rest, and every split-payout operation shall be atomic - fully completing or fully rolling back, with no partial payouts.	High	Pass: a simulated mid-transaction failure results in a complete rollback with no partial fund transfer, verified via the financial audit log. Fail: any partial or duplicated payout is recorded.	The platform moves real money owed to multiple independent third parties in a single transaction, so financial integrity is a non-negotiable trust requirement.

2. UML Use-Case Diagram



Actors: Shopper, Artisan Vendor (primary); Payment Gateway (secondary / external system). Relationships modelled: 3 «include» dependencies (Checkout to Authenticate User, Checkout to Process Payment, Manage Product Catalog to Authenticate User) and 2 «extend» dependencies (Apply Discount Coupon extends Checkout, Rate and Review Product extends Track Order).

3. Use-Case Flow Specification

Use case name: Checkout (Place Multi-Vendor Order and Process Payment)

Actors: Shopper (primary), Payment Gateway (secondary)

Preconditions

- The shopper's cart contains at least one item, from one or more vendors, each currently in stock.
- The shopper is authenticated, or authenticates during the flow (via the included Authenticate User use case).

Postconditions (on success)

- One sub-order per vendor is created, each set to status "Processing".
- Payment is captured from the shopper and split across vendor accounts, minus the 5% platform commission, per FR-001.
- The shopper receives an order confirmation; each vendor receives a new-order notification.

Main success scenario

1. Shopper selects "Proceed to Checkout" from the cart.
2. System verifies the shopper is authenticated; if not, it includes the Authenticate User use case.
3. System displays an order summary grouped by vendor, with itemized subtotal, shipping, and total.
4. Shopper selects a delivery address and payment method, and confirms the order.
5. System includes the Process Payment use case, sending a charge request to the Payment Gateway for the order total.
6. Payment Gateway authorizes and captures the payment, returning a success response.
7. System splits the captured amount across vendor sub-orders (item subtotal minus 5% commission) and credits each vendor account.
8. System creates one sub-order per vendor, sets status to "Processing", and generates a confirmation for the shopper and a notification for each vendor.
9. System displays the confirmation screen with per-vendor tracking references.

Alternate flow - A1: Payment declined

Triggered at step 6, if the Payment Gateway declines or fails to authorize the payment.

1. System displays a "Payment failed" message with the decline reason.
2. System keeps the cart intact and returns the shopper to the payment-method selection step (step 4).
3. Shopper may retry with a different payment method, or abandon checkout - no order or sub-orders are created, and the use case ends.